
Green RecSys: cuantificando el costo de carbono de la recomendación basada en grafos de conocimiento

Tomás Caillabet
Pontificia Universidad Católica de Chile
tcaillabet@uc.cl

Joaquín Rivas
Pontificia Universidad Católica de Chile
jrivas@uc.cl

Blanca Ortúzar
Pontificia Universidad Católica de Chile
bortuzar@uc.cl

Resumen

Los sistemas recomendadores basados en grafos de conocimiento (KG) alcanzan alta calidad de ranking, pero su huella de carbono casi nunca se reporta. En este trabajo medimos, con una metodología unificada, el trade-off entre calidad de recomendación y emisiones de CO₂eq en 13 modelos (2 triviales y 11 KG-aware que cubren 6 paradigmas algorítmicos) sobre 3 datasets públicos de distinta escala, registrando la energía de CPU, GPU y RAM de cada entrenamiento con CodeCarbon. Encontramos que (i) el trade-off es real: el mejor modelo (KGCL, contrastivo) supera al baseline MostPopular entre un 76 % y un 196 % en nDCG@10, pero emitiendo entre tres y cuatro órdenes de magnitud más CO₂; (ii) medido en nDCG por gramo de CO₂, lo trivial es imbatible (MostPopular supera al mejor modelo KG-aware entre 63 y 952 veces según el dataset), aunque al precio de una cobertura de catálogo casi nula; (iii) el costo varía hasta 38× entre modelos KG-aware de un mismo dataset y escala de forma aproximadamente lineal con las interacciones (×7,6–10,9 al pasar de 100 mil a 1 millón); y (iv) el ajuste de hiperparámetros tiene palancas de ahorro concretas: reducir el tamaño de los embeddings recorta hasta un 42 % de las emisiones con pérdidas menores de calidad, mientras que un learning rate mal elegido desperdicia un entrenamiento completo sin ahorrar energía. Publicamos el código, los resultados por corrida y las recomendaciones generadas para reproducir todos los experimentos.

1. Introducción

El costo ambiental del aprendizaje automático dejó de ser una preocupación marginal. La discusión nació con los grandes modelos fundacionales, cuyo entrenamiento puede emitir tanto CO₂ como varios automóviles durante su vida útil [1], y cristalizó en el llamado de la *Green AI* a reportar la eficiencia junto con la exactitud [2]. La recomendación no queda fuera de ese diagnóstico por usar modelos más pequeños: en producción, los sistemas de personalización se reentrenan continuamente sobre flujos de interacciones que no dejan de crecer y concentran la mayor parte de los ciclos de inferencia de IA en los centros de datos de las grandes plataformas [3], y mediciones de la propia comunidad muestran que los pipelines de recomendación basados en deep learning pueden multiplicar por cientos las emisiones de alternativas simples [4; 5]. Sin embargo, la familia de recomendadores basados en grafos de conocimiento (KG-aware), una de las líneas más activas de la última década, carece de una caracterización sistemática de su huella de carbono: los papers reportan nDCG y Recall, pero no cuánta energía costó cada punto de esas métricas.

Este trabajo aborda esa brecha con un estudio empírico guiado por tres preguntas:

Tabla 1: Datasets utilizados. La densidad es $|Inter|/(|\mathcal{U}| \cdot |\mathcal{I}|)$.

Dataset	Usuarios	Ítems	Interacciones	Densidad	Grafo de conocimiento
ML-100k	943	1 682	100 000	6,30 %	géneros de películas
ML-1M	6 040	3 706	1 000 209	4,47 %	géneros de películas
LastFM-2k	1 892	17 632	92 834	0,28 %	top-5 tags por artista

- ¿Cuánto CO₂ cuesta cada punto de calidad en los recomendadores KG-aware, y cómo se comparan con baselines triviales?
- ¿Cómo varía ese trade-off entre familias algorítmicas (embedding, propagación, GNN, multi-tarea, contrastivo, auto-supervisado, difusión) y entre datasets de distinta escala?
- ¿Cuál es el costo de carbono del ajuste de hiperparámetros, y qué perillas permiten ahorrar emisiones sin sacrificar calidad?

Nuestras contribuciones son: (1) un benchmark reproducible¹ de 13 modelos (11 KG-aware en 6 paradigmas) sobre 3 datasets, con medición de energía y CO₂ por corrida vía CodeCarbon [6] y evaluación unificada full-ranking; (2) un análisis del trade-off calidad-emisiones, incluyendo la métrica de eficiencia ambiental nDCG por gramo de CO₂ y la frontera de Pareto por dataset; y (3) un estudio del costo de fine-tuning mediante barridos OFAT.

2. Trabajo relacionado

Strubell et al. [1] cuantificaron el costo energético de entrenar modelos de NLP y Schwartz et al. [2] propusieron tratar la eficiencia como criterio de evaluación de primera clase, línea que se consolidó en herramientas como CodeCarbon [6], que estima el CO₂eq a partir de la energía consumida por CPU, GPU y RAM y del factor de emisión del país.

En recomendación, Spillo et al. [4] compararon el desempeño y la huella de carbono de algoritmos clásicos y neuronales, concluyendo que las mejoras de calidad suelen pagarse con emisiones desproporcionadas. Vente et al. [5] extendieron el análisis al ciclo completo de experimentación. Ninguno de estos trabajos cubre la familia KG-aware, que es precisamente donde la literatura concentra buena parte de las propuestas recientes; nuestro estudio la aborda de forma sistemática y por paradigma.

Los métodos KG-aware, por su parte, enriquecen la matriz usuario-ítem con tripletas (*head, relation, tail*). Se suelen agrupar en: *embedding*, que proyectan entidades y relaciones a espacios latentes (CKE [7], CFKG [8], KTUP [9]); *multi-tarea*, que alternan recomendación y completado del grafo (MKR [10]); *propagación*, que difunden preferencias por el grafo (RippleNet [11]); *GNN*, que agregan vecindarios con convoluciones sobre el grafo (KGCN [12], KGNN-LS [13], KGAT [14], KGIN [15]); y los más recientes basados en aprendizaje *auto-supervisado*: KGCL [16] (contrastivo), KGRec [17] (racionalización auto-supervisada) y DiffKG [18] (difusión generativa sobre el KG). Evaluamos representantes de todas estas familias.

3. Datasets

Usamos tres datasets públicos de distinta escala y densidad (Tabla 1): MovieLens-100k y MovieLens-1M [19], y Last.FM-2k (HetRec 2011) [20]. Cada uno se enriquece con un grafo de conocimiento construido desde sus propios metadatos: en MovieLens, tripletas ítem-has_genre-género (2 893 tripletas en ml-100k); en Last.FM-2k, tripletas artista-has_tag-etiqueta usando las 5 etiquetas sociales más frecuentes por artista. Las interacciones se binarizan como feedback implícito y se dividen 80/20 por usuario con semilla fija compartida (3633), de modo que todos los modelos ven exactamente la misma partición.

¹Código y resultados: <https://github.com/blancaortuzarm/RecSys-proyecto>

4. Metodología

Comparamos 2 baselines triviales implementados por nosotros (Random y MostPopular) contra 11 modelos KG-aware en 6 paradigmas: embedding (CKE, KTUP, CFKG), multi-tarea (MKR), propagación (RippleNet), GNN (KGCN, KGNN-LS, KGIN), contrastivo (KGCL), auto-supervisado (KGRec) y difusión (DiffKG).

Los modelos clásicos corren sobre RecBole 1.2.0 [21] a partir de archivos atómicos (.inter/.kg/.link) generados desde nuestra partición. KGIN, KGCL, KGRec y DiffKG no están en RecBole, por lo que se integraron desde SSLRec [22] ejecutado como subproceso con la misma partición convertida a su formato. Configuración homogénea salvo indicación: 15 épocas, embeddings de tamaño 64 (MKR usa 32 y batch 256 por memoria) y los learning rates por defecto de cada framework. Nótese que las familias contrastivo, auto-supervisado y difusión provienen exclusivamente de SSLRec, de modo que familia y biblioteca quedan parcialmente confundidas; en la Sección 5 usamos DiffKG, el único modelo SSLRec ejecutado en la misma GPU que RecBole, como punto de control para separar ambos efectos.

Todos los modelos se evalúan con protocolo full-ranking sobre el catálogo completo y $K = 10$: Precision@10, Recall@10, nDCG@10, MAP@10 y MRR, más tres métricas de cobertura del espacio de ítems: diversidad intra-lista (disimilitud promedio de categorías entre pares de ítems recomendados), cobertura de catálogo y novedad (auto-información, $-\log_2$ de la popularidad). Las métricas se calculan con un evaluador propio sobre el top- K exportado por cada modelo, lo que las hace comparables entre frameworks; para KGIN, KGCL y KGRec sólo se dispone del Recall@10 y nDCG@10 auto-reportados por SSLRec sobre la misma partición.

Cada entrenamiento (incluida su evaluación) se envuelve con el OfflineEmissionsTracker de CodeCarbon [6] con factor de emisión de Chile, registrando energía de CPU, GPU y RAM (kWh), potencias, duración y CO₂eq. Definimos la *eficiencia ambiental* de un modelo como

$$EA = \frac{\text{nDCG@10}}{\text{gramos de CO}_2\text{eq por entrenamiento}}. \quad (1)$$

Los experimentos corrieron en Google Colab: RecBole y DiffKG sobre GPU Tesla T4 (torch 2.11+cu128) y KGIN, KGCL y KGRec en sesiones sólo-CPU (Xeon 2,2 GHz, torch 2.11+cpu), lo que se discute como limitación en la Sección 7. El pipeline está organizado en un módulo común y notebooks independientes por familia que guardan resultados de forma incremental y reanudable (cada corrida agrega una fila con métricas completas, desglose energético, número de parámetros, configuración y hardware, y el top- K por usuario se persiste para poder recalcular métricas sin reentrenar).

5. Resultados

5.1. Calidad vs. CO₂: el trade-off existe y es de órdenes de magnitud

La Tabla 2 presenta los resultados completos en ml-100k y la Figura 1 el plano calidad-emisiones en los tres datasets. Siete de los once modelos KG-aware superan a MostPopular en nDCG@10 en ml-100k: KGCL (0,391), DiffKG (0,346), KGNN-LS y KGCN (0,310), CKE (0,308), KGRec (0,255) y MKR (0,246), frente a 0,222 del baseline. La ganancia máxima (+76 % de KGCL) cuesta, sin embargo, 654 mg de CO₂ contra 0,20 mg de MostPopular: más de tres órdenes de magnitud. En Last.FM-2k y ML-1M la brecha de calidad a favor de KGCL es aún mayor (+196 % y +91 %), pero el sobre costo crece a $\sim 1400\times$ y $\sim 8400\times$ respectivamente.

Un segundo hallazgo es que *gastar más no garantiza recomendar mejor*: RippleNet y MKR son los modelos más caros de ml-100k (1 535 y 1 081 mg) y quedan en la mitad inferior de calidad. La frontera de Pareto de ml-100k la forman CFKG, KGIN, KGNN-LS, DiffKG y KGCL: para cada nivel de presupuesto de carbono existe un modelo dominante, y ninguno de los dos más costosos pertenece a ella.

5.2. Generalización entre datasets y conclusiones por familia

La Tabla 5 del Anexo resume nDCG@10 y CO₂ en los tres datasets. KGCL es el mejor modelo en los tres, con nDCG@10 de 0,391, 0,242 y 0,391. A nivel de familia, los resultados son consistentes:

Tabla 2: Resultados en MovieLens-100k, ordenados por nDCG@10. Las métricas de KGCL, KGRec y KGIN (—) son las auto-reportadas por SSLRec sobre la misma partición. En **negrita**, el mejor valor por columna entre todos los modelos.

Modelo	Familia	Prec@10	Recall@10	nDCG@10	MRR	CO ₂ (mg)	nDCG/gCO ₂
KGCL	Contrastivo	—	0,215	0,391	—	654,4	0,60
DiffKG	Difusión	0,295	0,187	0,346	0,572	549,8	0,63
KGNN-LS	GNN	0,267	0,176	0,310	0,522	266,4	1,16
KGCN	GNN	0,267	0,176	0,310	0,522	270,5	1,15
CKE	Embedding	0,260	0,174	0,308	0,532	307,0	1,00
KGRec	Auto-superv.	—	0,114	0,255	—	430,3	0,59
MKR	Multi-tarea	0,213	0,134	0,246	0,442	1 081,4	0,23
MostPopular	Trivial	0,193	0,116	0,222	0,416	0,20	1 119,8
KGIN	GNN	—	0,091	0,213	—	214,9	0,99
CFKG	Embedding	0,181	0,108	0,201	0,364	170,6	1,18
RippleNet	Propagación	0,170	0,109	0,189	0,351	1 535,2	0,12
KTUP	Embedding	0,171	0,094	0,186	0,345	305,8	0,61
Random	Trivial	0,014	0,006	0,014	0,038	0,56	25,3



Figura 1: Trade-off calidad vs. CO₂ para los modelos KG-aware en los tres datasets (escala logarítmica en x). La línea punteada marca la calidad de MostPopular, cuyo CO₂ (<4 mg) queda fuera de escala. El *spread* indica la razón de emisiones entre el modelo KG-aware más caro y el más barato de cada dataset.

- **Contrastivo, difusión y auto-supervisado** (SSLRec) lideran la calidad en los tres datasets, pero con los costos más altos; en ML-1M, KGCL y KGRec emitieron 32 y 50 gramos por entrenamiento (horas de cómputo), un orden de magnitud sobre el resto.
- **GNN** ofrece el mejor equilibrio: en ml-100k, KGCN y KGNN-LS alcanzan el 79 % de la calidad de KGCL con el 41 % de sus emisiones. KGCN y KGNN-LS producen recomendaciones idénticas en nuestros datos: con grafos planos de un solo tipo de relación, la regularización por label smoothness no altera el ranking final.
- **Embedding** es la familia más barata; CKE resulta sorprendentemente competitivo (tercer mejor nDCG en ML-1M) y CFKG es el KG-aware más eficiente (1,18 nDCG/gCO₂ en ml-100k).
- **Multi-tarea y propagación** presentan la peor relación calidad-emisiones en los tres datasets: RippleNet, además, colapsa en ML-1M (0,052).
- **Trivial**: MostPopular queda octavo en calidad en ml-100k pero su costo prácticamente nulo lo hace, como se ve a continuación, imbatible en eficiencia.

Al pasar de ml-100k a ML-1M ($\times 10$ interacciones), las emisiones de los siete modelos RecBole crecen entre $\times 7,6$ (CFKG) y $\times 10,9$ (KGNN-LS), con mediana $\times 9,4$: el costo escala de forma aproximadamente lineal con el volumen de interacciones, y la dispersión *entre* modelos KG-aware de un mismo dataset crece de $9\times$ (ml-100k) a $12\times$ (LastFM-2k) y $38\times$ (ML-1M); la Figura 4 del Anexo desglosa las emisiones modelo a modelo. Elegir bien el modelo importa cada vez más a medida que crece el dataset.

En cuanto al desglose energético: en los modelos KG-aware ejecutados con GPU, esta concentra entre el 65 % y el 81 % de la energía total (máximo en RippleNet); el resto se reparte entre CPU y RAM. Los baselines triviales consumen cantidades despreciables ($\sim 10^{-6}$ kWh en ml-100k).

Dado que las familias auto-supervisadas provienen sólo de SSLRec, cabe preguntarse si su mayor costo es del método o de la implementación. Tres observaciones sugieren que domina el método. Primero, *dentro* de RecBole (misma biblioteca, misma GPU) el costo ya varía $9\times$ en ml-100k (CFKG 171 mg vs. RippleNet 1 535 mg): la elección de algoritmo explica órdenes de dispersión sin cambiar de framework. Segundo, DiffKG, el único modelo SSLRec ejecutado en la misma GPU T4, emite 550 mg y entrena en 119 s en ml-100k, dentro del rango de los modelos RecBole (43–295 s; 171–1 535 mg), pese a que nuestra medición de SSLRec incluye el subproceso completo (conversión de datos y carga incluidas): no observamos un sobre costo estructural de la biblioteca cuando el hardware coincide. Tercero, el sobre costo de KGCL y KGRec es consistente con su régimen de entrenamiento (la augmentación contrastiva y la racionalización requieren 580 y 381 s en ml-100k, frente a 82 s de CKE), amplificado por haber corrido en CPU; sus 32 y 50 g en ML-1M deben leerse, por tanto, como cotas superiores. En síntesis, la jerarquía de costos refleja principalmente al método y su régimen de entrenamiento, no a la biblioteca que lo implementa.

5.3. Eficiencia ambiental

La última columna de la Tabla 2 resume la eficiencia ambiental en ml-100k; en el Anexo, la Figura 3 la muestra por familia y la Figura 5 por modelo. MostPopular alcanza 1 119,8 puntos de nDCG por gramo de CO_2 ; el mejor KG-aware (CFKG, 1,18) queda 952 veces por debajo, y el mejor en calidad (KGCL, 0,60) casi 1 900 veces. La brecha se repite en LastFM-2k ($63\times$) y ML-1M ($243\times$). La lectura no es que los modelos KG-aware “no valgan la pena”, sino que bajo una restricción ambiental estricta (por ejemplo, reentrenos frecuentes a gran escala) un baseline de popularidad bien afinado es extraordinariamente difícil de justificar reemplazar, y que la ganancia de calidad de los modelos complejos debe reportarse junto a su costo. La Sección 5.4 matiza este dominio: la eficiencia de MostPopular viene acompañada de una cobertura de catálogo casi nula.

5.4. Diversidad, cobertura y novedad: el costo oculto de lo trivial

La Tabla 3 completa el cuadro con las métricas de cobertura del espacio de ítems. La cobertura de catálogo revela el fallo de MostPopular: recomienda apenas el 3,3 % del catálogo en ml-100k y el 0,2 % en LastFM-2k (1,8 % en ML-1M), mientras que en ml-100k los modelos KG-aware cubren entre el 4 % y el 42 % (y hasta el 48 % en ML-1M, donde KGCL y KGNN-LS lideran). Su dominio en eficiencia ambiental debe leerse con esta salvedad: es un sistema que concentra toda su calidad en la cabeza de la distribución de popularidad.

Entre los KG-aware, CCFG maximiza la diversidad intra-lista en ml-100k (0,429) y DiffKG la cobertura (0,421) y la novedad (25,9), seguido por RippleNet (0,299 y 23,0), que sin embargo es el modelo más caro. En LastFM-2k, MKR y RippleNet lideran diversidad (0,589 y 0,577) y RippleNet novedad (62,7), nuevamente con costos altos. En general, calidad, diversidad y sostenibilidad rara vez coinciden en un mismo modelo; DiffKG es la excepción parcial en ml-100k, combinando el segundo mejor nDCG con la mejor cobertura y novedad entre los KG-aware, a un costo intermedio.

6. Análisis de parámetros: el costo de carbono del fine-tuning

El ajuste de hiperparámetros multiplica los entrenamientos y, por tanto, las emisiones, pero rara vez se contabiliza. Ejecutamos en ml-100k un barrido OFAT (un factor a la vez, 3 valores por hiperparámetro) sobre los cuatro modelos de la familia embedding/multi-tarea (CKE, KTUP, MKR, CCFG): 27 corridas y 14,1 g de CO_2 en total, cerca de cuatro veces menos que el grid completo 3^3 por modelo, que proyectamos en 55,3 g a partir del costo medio por corrida. La Tabla 4 resume los efectos principales.

La Figura 2 visualiza los dos efectos dominantes del barrido, de los que emergen tres lecciones prácticas. **(1) El learning rate es la perilla más sensible en calidad y la más barata de equivocarse:** variarlo casi no cambia el CO_2 por corrida ($\pm 5\%$), pero un valor muy bajo colapsa la calidad (-76% en CKE, -59% en KTUP), desperdiciando la energía de un entrenamiento completo; y el default puede ser subóptimo (en KTUP, subirlo a 10^{-2} regala $+53\%$ de nDCG). **(2) El tamaño del**

Tabla 3: Diversidad intra-lista, cobertura de catálogo y novedad ($-\log_2$ de popularidad) para los modelos con evaluación completa. KGCL, KGRec y KGIN no aparecen porque estas métricas requieren las listas top- K por usuario, que SSLRec no exporta (sólo auto-reporta Recall y nDCG); DiffKG sí se incluye porque sus recomendaciones se volcaron y evaluaron con nuestro propio evaluador. En **negrita**, el máximo por columna entre los modelos KG-aware.

Modelo	ML-100k			LastFM-2k		
	Divers.	Cobert.	Novedad	Divers.	Cobert.	Novedad
DiffKG	0,399	0,421	25,9	0,511	0,250	55,3
KGNN-LS	0,410	0,271	21,8	0,445	0,045	39,7
KGCN	0,410	0,271	21,8	0,445	0,045	39,7
CKE	0,412	0,118	17,6	0,435	0,039	32,7
MKR	0,422	0,137	21,5	0,589	0,014	43,2
CFKG	0,429	0,038	14,5	0,489	0,004	25,4
RippleNet	0,409	0,299	23,0	0,577	0,199	62,7
KTUP	0,386	0,078	15,4	0,490	0,011	29,0
MostPopular	0,404	0,033	14,4	0,410	0,002	24,4
Random	0,506	0,995	58,0	0,929	0,705	100,1

Tabla 4: Efectos del barrido OFAT en ml-100k: cambio porcentual de nDCG@10 y de CO₂ respecto de la configuración base de cada modelo. Se listan las perillas con efectos más informativos.

Modelo	Cambio	Δ nDCG@10	Δ CO ₂
CKE	learning rate $10^{-3} \rightarrow 10^{-4}$	-76,0 %	-4,7 %
CKE	embedding 64 $\rightarrow$ 128	+4,6 %	+89,1 %
CKE	embedding 64 $\rightarrow$ 32	-7,6 %	-41,7 %
CKE	embedding KG 64 $\rightarrow$ 32	+1,5 %	-31,2 %
KTUP	learning rate $10^{-3} \rightarrow 10^{-2}$	+53,0 %	-1,3 %
KTUP	embedding 64 $\rightarrow$ 128	+51,6 %	+46,3 %
KTUP	embedding 64 $\rightarrow$ 32	-31,9 %	-24,7 %
MKR	embedding 32 $\rightarrow$ 16	+0,4 %	-30,1 %
CFKG	embedding 64 $\rightarrow$ 128	+29,9 %	+7,0 %
CFKG	loss BPR $\rightarrow$ TransE	+18,8 %	+7,9 %
CFKG	learning rate $10^{-3} \rightarrow 10^{-2}$	+17,4 %	+3,4 %

embedding es la palanca principal de energía, y su retorno depende del modelo: duplicarlo en CKE cuesta +89 % de CO₂ por sólo +4,6 % de calidad, mientras que en KTUP el mismo cambio sí paga (+51,6 % de nDCG). En modelos pequeños sobre GPU (CFKG), el costo fijo de la corrida domina y duplicar el embedding apenas sube las emisiones (+7 %). **(3) Existen ahorros gratuitos:** reducir el embedding del KG en CKE recorta un 31 % de emisiones *mejorando* marginalmente la calidad, y en MKR bajar el embedding de 32 a 16 ahorra un 30 % sin pérdida alguna.

El barrido usó una configuración experimental con un grafo de conocimiento distinto al de la Sección 3 (el KG derivado de Freebase que RecBole asocia por defecto a ml-100k), por lo que sus niveles absolutos de nDCG no son comparables con los de la Tabla 2; las sensibilidades relativas y los costos energéticos, que dependen del tamaño del modelo y no del grafo, sí son informativos.

7. Discusión y limitaciones

Los resultados admiten una lectura práctica como guía de selección bajo un presupuesto de carbono. Si la restricción ambiental es estricta y el sesgo de popularidad es tolerable, MostPopular o un embedding ligero como CFKG ofrecen una eficiencia inalcanzable para el resto, con la salvedad de cobertura discutida en la Sección 5.4. Con un presupuesto moderado, las GNN (KGCN, KGNN-LS) capturan cerca del 80 % de la calidad del mejor modelo a menos de la mitad de su costo, y DiffKG agrega la mejor cobertura y novedad a un costo intermedio. La máxima calidad (KGCL) exige asumir entre tres y cuatro órdenes de magnitud más emisiones que lo trivial. Esta decisión, además, pesa más a medida que crece el dato: con 10 veces más interacciones, la dispersión de costos entre modelos KG-aware llega a $38\times$.

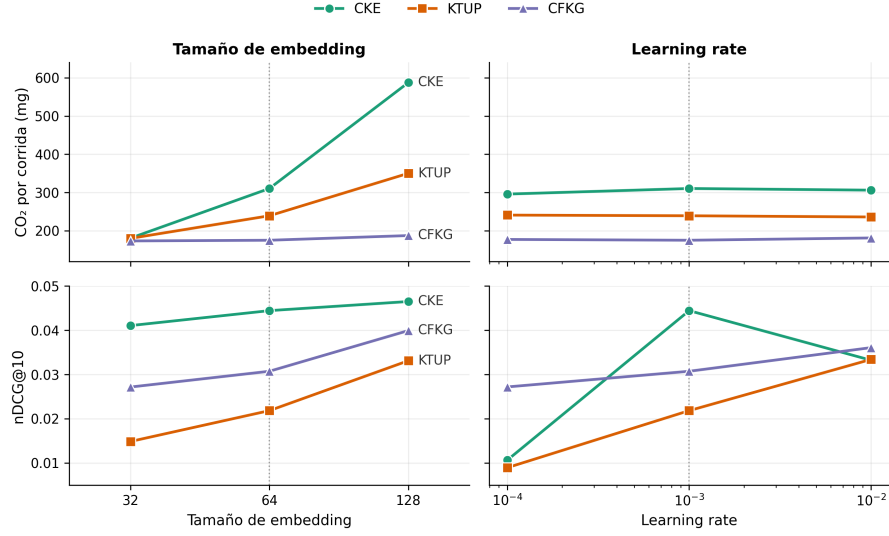


Figura 2: Efecto del tamaño de embedding (izquierda) y del learning rate (derecha) sobre las emisiones (arriba) y la calidad (abajo) para los tres modelos de la familia embedding en ml-100k. La línea punteada marca la configuración base. El CO₂ escala con el tamaño del modelo (salvo en CFKG, cuyo costo fijo de corrida en GPU domina) pero es insensible al learning rate, cuya mala elección colapsa la calidad desperdiciando un entrenamiento completo. CKE y KTUP se ejecutaron en CPU y CFKG en GPU, por lo que los niveles absolutos de CO₂ no son comparables entre modelos.

Una segunda línea de discusión es el grafo mismo. Nuestros KGs usan un único tipo de relación por dataset (géneros o etiquetas), mientras que la literatura suele emplear grafos multi-relacionales derivados de bases de conocimiento abiertas, con decenas de tipos de relación. Es plausible que grafos más ricos favorezcan relativamente a los modelos que explotan la semántica relacional (KTUP, MKR, KGIN); de hecho, con una sola relación la regularización de KGNN-LS no altera el ranking de KGCL. Al mismo tiempo, un grafo mayor eleva el costo de entrenamiento, por lo que cuantificar ese doble efecto sobre la frontera calidad–emisiones es una extensión natural de este trabajo.

En cuanto al alcance: DiffKG no se evaluó en ML-1M, y para KGIN, KGCL y KGRec se dispone de Recall y nDCG auto-reportados sobre la misma partición, sin las métricas de diversidad y cobertura. Reportamos, además, el costo de *entrenamiento más evaluación*: en producción, la inferencia acumulada sobre millones de consultas puede alterar el balance, y medir sobre múltiples semillas para cuantificar la varianza queda como trabajo futuro.

8. Conclusiones

Presentamos la primera caracterización sistemática del trade-off calidad–carbono en recomendadores basados en grafos de conocimiento: 13 modelos, 6 paradigmas y 3 datasets con medición de energía por corrida. Nuestros resultados muestran que (1) el trade-off es real y de órdenes de magnitud: la mejor calidad (KGCL) cuesta entre tres y cuatro órdenes más CO₂ que un baseline trivial; (2) siete modelos KG-aware superan a MostPopular en calidad en ml-100k, y las GNN logran casi la calidad de los contrastivos a menos de la mitad de su costo, definiendo puntos intermedios atractivos de la frontera de Pareto; (3) en eficiencia ambiental (nDCG/gCO₂), lo trivial domina por 63–952× según el dataset, aunque con una cobertura de catálogo casi nula (3.3 % o menos) frente a coberturas de hasta 42–48 % de los mejores modelos KG-aware; (4) el costo escala aproximadamente lineal con las interacciones y la dispersión entre modelos KG-aware llega a 38×, por lo que la elección de modelo es también una decisión ambiental; y (5) el fine-tuning ofrece palancas concretas de ahorro (embeddings pequeños, hasta −42 % de emisiones) y riesgos concretos de derroche (learning rates mal elegidos).

Todo el estudio, incluido el barrido de hiperparámetros, emitió del orden de 160 g de CO₂eq en corridas exitosas, aproximadamente lo que un automóvil promedio emite en un kilómetro: medir el

costo ambiental de la experimentación es barato, y no hacerlo deja fuera una dimensión completa de la evaluación. Proponemos que la eficiencia ambiental se reporte como métrica estándar en recomendación, junto a la calidad de ranking. El código, los resultados por corrida y las recomendaciones generadas están disponibles en <https://github.com/blancaortuzarm/RecSys-proyecto> para reproducir todos los experimentos.

Referencias

- [1] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 3645–3650, 2019.
- [2] Roy Schwartz, Jesse Dodge, Noah A. Smith, and Oren Etzioni. Green AI. *Communications of the ACM*, 63(12):54–63, 2020.
- [3] Udit Gupta, Carole-Jean Wu, Xiaodong Wang, Maxim Naumov, Brandon Reagan, David Brooks, Bradford Cottel, Kim Hazelwood, Mark Hempstead, Bill Jia, Hsien-Hsin S. Lee, Andrey Malevich, Dheevatsa Mudigere, Mikhail Smelyanskiy, Liang Xiong, and Xuan Zhang. The architectural implications of facebook’s DNN-based personalized recommendation. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 488–501, 2020.
- [4] Giuseppe Spillo, Allegra De Filippo, Cataldo Musto, Michela Milano, and Giovanni Semeraro. Towards sustainability-aware recommender systems: Analyzing the trade-off between algorithms performance and carbon footprint. In *Proceedings of the 17th ACM Conference on Recommender Systems (RecSys)*, pages 856–862, 2023.
- [5] Tobias Vente, Lukas Wegmeth, Alan Said, and Joeran Beel. From clicks to carbon: The environmental toll of recommender systems. In *Proceedings of the 18th ACM Conference on Recommender Systems (RecSys)*, pages 580–590, 2024.
- [6] Benoît Courty, Victor Schmidt, Sasha Luccioni, Goyal Kamal, Marion Coutarel, Boris Feld, Jérémy Léval, et al. mlco2/codecarbon: v2.4.1. Zenodo, <https://doi.org/10.5281/zenodo.11171501>, 2024.
- [7] Fuzheng Zhang, Nicholas Jing Yuan, Defu Lian, Xing Xie, and Wei-Ying Ma. Collaborative knowledge base embedding for recommender systems. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 353–362, 2016.
- [8] Qingyao Ai, Vahid Azizi, Xu Chen, and Yongfeng Zhang. Learning heterogeneous knowledge base embeddings for explainable recommendation. *Algorithms*, 11(9):137, 2018.
- [9] Yixin Cao, Xiang Wang, Xiangnan He, Zikun Hu, and Tat-Seng Chua. Unifying knowledge graph learning and recommendation: Towards a better understanding of user preferences. In *Proceedings of The Web Conference (WWW)*, pages 151–161, 2019.
- [10] Hongwei Wang, Fuzheng Zhang, Miao Zhao, Wenjie Li, Xing Xie, and Minyi Guo. Multi-task feature learning for knowledge graph enhanced recommendation. In *Proceedings of The Web Conference (WWW)*, pages 2000–2010, 2019.
- [11] Hongwei Wang, Fuzheng Zhang, Jialin Wang, Miao Zhao, Wenjie Li, Xing Xie, and Minyi Guo. Ripplenet: Propagating user preferences on the knowledge graph for recommender systems. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 417–426, 2018.
- [12] Hongwei Wang, Miao Zhao, Xing Xie, Wenjie Li, and Minyi Guo. Knowledge graph convolutional networks for recommender systems. In *Proceedings of The Web Conference (WWW)*, pages 3307–3313, 2019.
- [13] Hongwei Wang, Fuzheng Zhang, Mengdi Zhang, Jure Leskovec, Miao Zhao, Wenjie Li, and Zhongyuan Wang. Knowledge-aware graph neural networks with label smoothness regularization for recommender systems. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 968–977, 2019.

- [14] Xiang Wang, Xiangnan He, Yixin Cao, Meng Liu, and Tat-Seng Chua. KGAT: Knowledge graph attention network for recommendation. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 950–958, 2019.
- [15] Xiang Wang, Tinglin Huang, Dingxian Wang, Yancheng Yuan, Zhenguang Liu, Xiangnan He, and Tat-Seng Chua. Learning intents behind interactions with knowledge graph for recommendation. In *Proceedings of The Web Conference (WWW)*, pages 878–887, 2021.
- [16] Yuhao Yang, Chao Huang, Lianghao Xia, and Chenliang Li. Knowledge graph contrastive learning for recommendation. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, pages 1434–1443, 2022.
- [17] Yuhao Yang, Chao Huang, Lianghao Xia, and Chunzhen Huang. Knowledge graph self-supervised rationalization for recommendation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pages 3046–3056, 2023.
- [18] Yangqin Jiang, Yuhao Yang, Lianghao Xia, and Chao Huang. DiffKG: Knowledge graph diffusion model for recommendation. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining (WSDM)*, pages 313–321, 2024.
- [19] F. Maxwell Harper and Joseph A. Konstan. The movielens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):19:1–19:19, 2015.
- [20] Iván Cantador, Peter Brusilovsky, and Tsvi Kuflik. Second workshop on information heterogeneity and fusion in recommender systems (HetRec2011). In *Proceedings of the 5th ACM Conference on Recommender Systems (RecSys)*, pages 387–388, 2011.
- [21] Wayne Xin Zhao, Shanlei Mu, Yupeng Hou, Zihan Lin, Yushuo Chen, Xingyu Pan, Kaiyuan Li, Yujie Lu, Hui Wang, Changxin Tian, Yingqian Min, Zhichao Feng, Xinyan Fan, Xu Chen, Pengfei Wang, Wendi Ji, Yaliang Li, Xiaoling Wang, and Ji-Rong Wen. Recbole: Towards a unified, comprehensive and efficient framework for recommendation algorithms. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 4653–4664, 2021.
- [22] Xubin Ren, Lianghao Xia, Yuhao Yang, Wei Wei, Tianle Wang, Xuheng Cai, and Chao Huang. SSLRec: A self-supervised learning framework for recommendation. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining (WSDM)*, pages 567–575, 2024.

A. Anexo: tablas y figuras adicionales

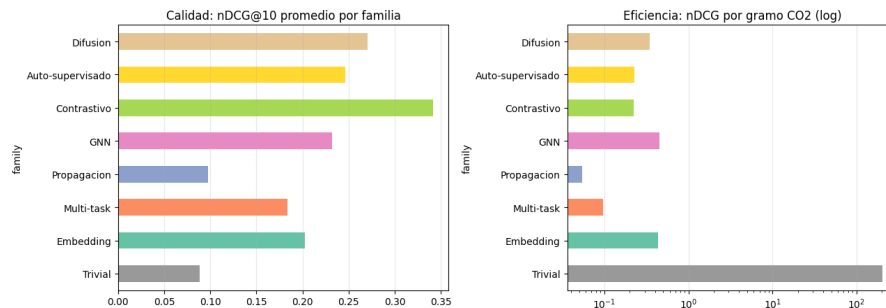


Figura 3: Panorama por familia en ml-100k: nDCG@10 promedio (izquierda) y eficiencia ambiental promedio en escala logarítmica (derecha). Las familias auto-supervisadas maximizan calidad; lo trivial domina la eficiencia por órdenes de magnitud.

Tabla 5: Generalización entre datasets: nDCG@10 y CO₂ por entrenamiento (mg). DiffKG no se evaluó en ML-1M. En **negrita**, el mejor nDCG@10 por dataset.

Modelo	ML-100k		LastFM-2k		ML-1M	
	nDCG	CO ₂	nDCG	CO ₂	nDCG	CO ₂
KGCL	0,391	654,4	0,242	3 784,0	0,391	32 046,4
DiffKG	0,346	549,8	0,196	3 416,5	—	—
KGNN-LS	0,310	266,4	0,145	820,7	0,273	2 911,6
KGCN	0,310	270,5	0,145	826,4	0,273	2 874,8
CKE	0,308	307,0	0,152	324,9	0,302	2 412,5
KGRec	0,255	430,3	0,155	1 858,9	0,329	49 645,0
MKR	0,246	1 081,4	0,075	1 967,0	0,231	10 166,8
MostPopular	0,222	0,20	0,082	2,8	0,204	3,8
KGIN	0,213	214,9	0,110	711,9	0,307	6 513,1
CFKG	0,201	170,6	0,072	383,3	0,287	1 303,8
RippleNet	0,189	1 535,2	0,052	1 444,9	0,052	11 957,0
KTUP	0,186	305,8	0,056	1 279,1	0,260	3 119,9
Random	0,014	0,56	0,001	9,0	0,010	8,8

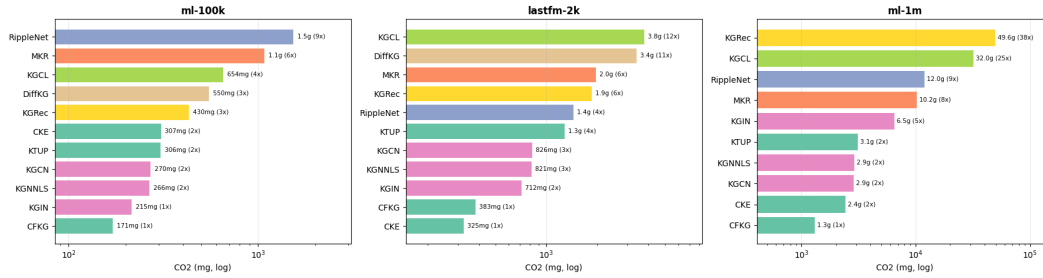


Figura 4: Huella de carbono por entrenamiento de cada modelo KG-aware en los tres datasets (escala logarítmica). La etiqueta indica las emisiones y la razón respecto del modelo KG-aware más barato del dataset: la dispersión crece de 9× en ml-100k a 12× en LastFM-2k y 38× en ML-1M.

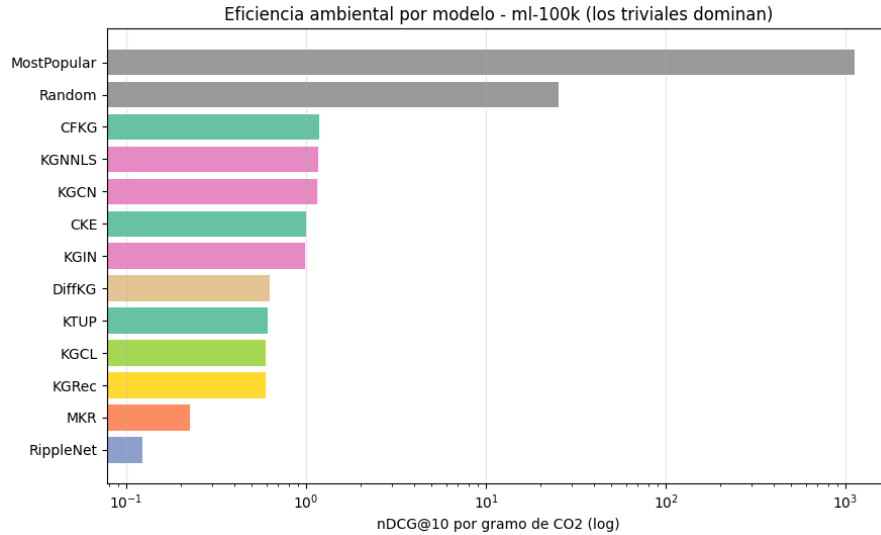


Figura 5: Eficiencia ambiental (nDCG@10 por gramo de CO₂eq, escala logarítmica) por modelo en ml-100k. Los baselines triviales superan a todo modelo KG-aware por dos a tres órdenes de magnitud; entre los KG-aware, CFKG, KGNN-LS y KGCN son los más eficientes.

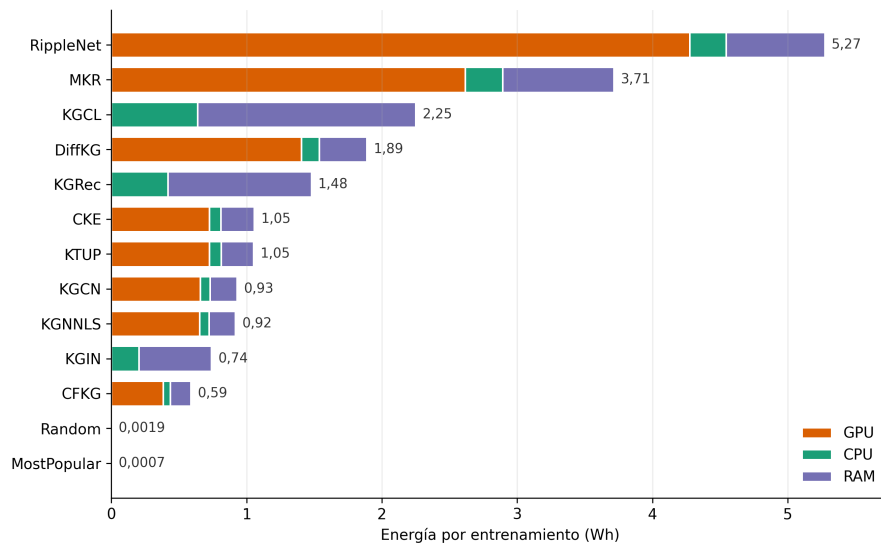


Figura 6: Desglose de la energía por componente (GPU, CPU y RAM) en ml-100k, en Wh por entrenamiento. En los modelos ejecutados con GPU, esta concentra entre el 65 % y el 81 % del consumo; KGCL, KGRec y KGIN corrieron en sesiones sólo-CPU, por lo que sus barras no incluyen GPU. Los baselines triviales son prácticamente nulos.